

01 — Gaussian Process Fundamentals

What you'll learn: what a GP is mathematically, how conditioning produces a posterior with a predictive mean and variance, and how hyperparameters are learned.

Prereqs: multivariate Gaussian distributions; basic linear algebra.

Why it matters here: this is the machinery behind every later doc and the `gp_velocity` module.

1. From a single function to a distribution over functions

Ordinary regression fits one function $f(x)$ to data. A GP instead maintains a *distribution over all

plausible functions* and updates it with data. Formally:

$$f(x) \sim \text{GP}(m(x), k(x, x'))$$

- $m(x)$ — the **mean function**, our prior guess for the output before seeing data. Usually $m \equiv 0$

after centering.

- $k(x, x')$ — the **covariance (kernel) function**, the heart of the GP. It returns how strongly

$f(x)$ and $f(x')$ covary. Typically: inputs close together $\rightarrow$ outputs strongly correlated; far apart $\rightarrow$ nearly independent.

The defining property: for **any** finite set of inputs $X = \{x_1, \dots, x_n\}$, the vector of outputs $f = [f(x_1), \dots, f(x_n)]^T$ is **jointly Gaussian**:

$$f \sim N(m, K), \quad \text{where } K[i, j] = k(x_i, x_j)$$

K is the $n \times n$ **covariance (Gram) matrix**. Everything a GP does is multivariate-Gaussian algebra on K .

2. Sampling from the prior (intuition)

Before any data, you can draw functions from the GP: pick a dense grid of inputs, build K , and sample $f \sim N(0, K)$. With a smooth kernel you get smooth wiggly curves; with a rough kernel you get

jagged ones. The kernel's **lengthscale** controls how fast the curves wiggle, its **variance** controls their amplitude. (See doc 02 for the menu.)

This is the prior: "here are the kinds of functions I believe in." Data then narrows it down.

3. Conditioning on data $\rightarrow$ the posterior (the key step)

Suppose we observe noisy data $y = f(X) + \varepsilon$, with $\varepsilon \sim N(0, \sigma_n^2 I)$ (i.i.d. observation noise of

variance σ_n^2). We want predictions at new test inputs X^* . Stack training and test outputs; they are jointly Gaussian:

$$\begin{bmatrix} y \\ f^* \end{bmatrix} \sim N \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} K(X, X) + \sigma_n^2 I & K(X, X^*) \\ K(X^*, X) & K(X^*, X^*) \end{bmatrix} \right)$$

Applying the standard Gaussian conditioning formula gives the **posterior** $f^* \mid X, y, X^*$, which is again Gaussian with:

$$\begin{aligned}\text{mean}^* &= K(X^*, X) [K(X, X) + \sigma_n^2 I]^{-1} y \\ \text{cov}^* &= K(X^*, X^*) - K(X^*, X) [K(X, X) + \sigma_n^2 I]^{-1} K(X, X^*)\end{aligned}$$

Read these two lines carefully — they are the whole of GP prediction:

- **Predictive mean** is a weighted average of the training targets y , where the weights come from

how similar (per the kernel) each test point is to the training points.

- **Predictive covariance** starts at the prior $K(X^*, X^*)$ and is reduced by a term that grows with how much the data constrains the test point. Near data $\rightarrow$ big reduction $\rightarrow$ small variance. Far

from data $\rightarrow$ little reduction $\rightarrow$ variance returns to the prior. This growth-away-from-data is exactly the property we exploit to bridge DVL dropouts (doc 04).

Let $\alpha = [K + \sigma_n^2 I]^{-1} y$ (computed once). Then for a single test point x^* :

$$\begin{aligned}\text{mean}(x^*) &= k(x^*, X) \cdot \alpha \\ \text{var}(x^*) &= k(x^*, x^*) - k(x^*, X) [K + \sigma_n^2 I]^{-1} k(X, x^*)\end{aligned}$$

4. A tiny worked example (1-D, by hand intuition)

Say we use the squared-exponential kernel $k(x, x') = \exp(-(x-x')^2/2)$ (unit variance, unit lengthscale)

and observe two noise-free points: $f(0)=0$, $f(2)=1$.

- At $x^*=0$ (on a data point): the posterior mean is 0 and variance ≈ 0 — the GP is certain.
- At $x^*=1$ (between the two points): the mean interpolates to roughly 0.5, with **small** variance

— it is "boxed in" by neighbours on both sides.

- At $x^*=5$ (far to the right of all data): the mean decays back toward the prior mean 0, and the

variance climbs back toward the prior variance 1 — the GP admits "I don't know out here."

That qualitative behaviour — confident interpolation, honest extrapolation — is the entire reason GPs

are useful for filling sensor gaps.

5. Hyperparameters and the marginal likelihood

The kernel has free **hyperparameters** θ (lengthscale, variance) plus the noise σ_n^2 . We don't guess them; we learn them by maximizing the **log marginal likelihood** of the observed data:

$$\log p(y \mid X, \theta) = -\frac{1}{2} y^T (K\theta + \sigma_n^2 I)^{-1} y - \frac{1}{2} \log |K\theta + \sigma_n^2 I| - (n/2) \log 2\pi$$

- The first term rewards **fitting the data** (small when y is explained well).
- The second term, $-\frac{1}{2} \log |K + \sigma_n^2 I|$, is a **complexity penalty** (Occam's razor) — it punishes overly

flexible kernels. This automatic trade-off is one of the GP's most attractive features: it resists overfitting without a separate validation set.

In practice we maximize this with gradient ascent (e.g. Adam). Libraries like GPyTorch

compute the
gradients by autodiff — see `implementation/02`.

6. What you get out, and why it's special

A trained GP returns, for any query input, `(mean, variance)` — a prediction *with calibrated uncertainty*. Most regressors give only a point estimate. The variance is what a Kalman filter needs

as its measurement-noise term R , which is precisely how we will couple the GP to the existing EKF

(doc 04,
`implementation/00`).

7. The catch (motivates docs 03, 05, 08)

The posterior requires solving $[K + \sigma_n^2 I]^{-1}$ for an $n \times n$ matrix — $O(n^3)$ to factorize and $O(n^2)$ memory. With n = thousands of DVL samples this is the central scalability problem. Three

escapes: keep n small (sliding window), use the **state-space/Kalman** form for $O(n)$

(doc 05), or use **sparse/inducing-point**

approximations (doc 08).

Key takeaways

- GP = mean function + kernel; any finite set of outputs is jointly Gaussian.
- Conditioning on data gives a Gaussian **posterior**: predictive **mean** (data-weighted) and **variance** (small near data, grows away — great for gap-filling).
- Hyperparameters are learned by maximizing the **marginal likelihood**, which auto-balances fit vs complexity.
- The cost is $O(n^3)$; later docs show how to dodge it.

Further reading

- Rasmussen & Williams, GPML, ch. 2 (regression) — the source for all formulas above.
- Next: 02 — Kernels and covariance functions.